OptiQueue: Distributed Task Scheduling & Execution Engine

System Architecture, First-Principles Technical Dictionary & Interview Masterclass

Author: Kaushtubha (Core Maintainer & Systems Architect) | **Live Demo:** https://optiqueue.onrender.com | **Stack:** Java 17, Spring Boot, Redis, PostgreSQL, Render, Neon, Upstash

1. Executive Overview & High-Level Architecture

OptiQueue is a production-grade distributed task execution engine engineered to guarantee fault-tolerant, prioritized, at-least-once job execution across distributed worker nodes. It natively incorporates Earliest-Deadline-First (EDF) dynamic scheduling, atomic distributed leasing via Redis, transactional state transitions in PostgreSQL, exponential backoff with full decorrelated jitter, and automated supervisor-driven failover detection.

Core Subsystem Topology

- **Ingestion Tier:** Spring Boot REST endpoints validate authorization via `X-API-Key`, enforce business-level `idempotencyKey` constraints, apply sliding-window rate limiting, and write immutable state to PostgreSQL.
- **Coordination Tier:** An in-memory concurrent Min-Heap priority queue sorted by EDF operates alongside Upstash Redis distributed locks (using atomic Lua scripts) to eliminate race conditions and double-processing.
- **Execution Tier:** Bounded worker thread pools consume jobs, route execution to registered domain handlers (`EMAIL_SEND` , `WEBHOOK_DISPATCH` , `REPORT_GEN`), and record telemetry.
- **Supervisory Watchdog:** The `FailoverDetector` and `HeartbeatService` constantly monitor cluster health; if a worker node crashes or network-partitions, orphaned tasks are automatically reassigned.

Architectural Trade-offs: Why Redis + PostgreSQL?

System Component	Role in OptiQueue	Engineering Justification
Neon PostgreSQL 16	System of Record (ACID Ledger)	Crash resilience, immutable execution audit logs, optimistic locking versioning (fencing tokens), durable dead-letter storage.
Upstash Redis 8.4	Coordination & Fast Ephemeral State	Sub-millisecond distributed locks with atomic lease renewal, sliding-window rate limiting (Sorted Sets), worker heartbeat registry.
Min-Heap Priority Queue	In-Memory Dispatching Engine	O(log n) task insertion and O(1) peek priority dispatching based on SLA deadlines.

2. First-Principles Technical Dictionary (Every Definition Explained)

1. Idempotency & Idempotency Key

Definition: An operation is idempotent if executing it multiple times yields the exact same outcome and side-effects as executing it once: `f(f(x)) = f(x)`.

OptiQueue Mechanism: Clients submit requests with a unique UUID `idempotencyKey`. PostgreSQL enforces a unique constraint on `idempotency_key`. If a duplicate network request arrives, OptiQueue catches the unique constraint violation and returns the existing job status without re-executing business side-effects.

2. Earliest-Deadline-First (EDF) Scheduling & Min-Heap

Definition: EDF is an optimal dynamic priority scheduling algorithm where tasks with the earliest deadline are prioritized for CPU/worker allocation. A Min-Heap is a binary tree where each node is smaller than or equal to its children.

OptiQueue Mechanism: `JobPriorityQueue.java` maintains an in-memory Min-Heap. The comparison function evaluates:

```
Score = (Priority × 10^12) + DeadlineEpochMillis
```

This ensures high-priority jobs execute first, while tasks approaching SLA expiration preempt lower-priority background tasks.

3. Distributed Lock & Lease TTL

Definition: Mutual exclusion across independent machines that do not share memory. A lease TTL (Time-To-Live) guarantees the lock is released if the holder crashes.

OptiQueue Mechanism: `DistributedLockService.java` acquires locks via `SET key worker_uuid NX PX 30000`. Lock release is performed via an atomic Lua script that verifies the caller owns the UUID before deleting the key, preventing accidental unlocks.

4. Fencing Tokens & Split-Brain Mitigation

Definition: A monotonically increasing token that prevents "zombie" workers (paused by Stop-the-World GC pauses) from committing writes after their lease expired.

OptiQueue Mechanism: PostgreSQL uses optimistic locking: `UPDATE jobs SET status = 'COMPLETED' WHERE id = :id AND version = :fencingToken`. If another worker took over during a pause, the version incremented and the stale write is rejected.

5. Heartbeat & Watchdog Failover Detection

Definition: A periodic ping sent by worker nodes to confirm liveness. A watchdog thread monitors heartbeats and triggers failover when nodes fail to report.

OptiQueue Mechanism: `HeartbeatService.java` writes to Redis key `worker:heartbeat:{workerId}` with a 15s TTL every 5s. `FailoverDetector.java` sweeps every 10s for `IN_PROGRESS` tasks whose worker key is missing, resetting them to `PENDING` for other workers.

6. Exponential Backoff with Decorrelated Jitter

Definition: An algorithmic retry strategy where delay grows exponentially with randomized noise to prevent the Thundering Herd Problem.

Mathematical Formula:

```
t_backoff = min(t_max, t_base × 2^retryCount) + rand(0, jitter)
```

OptiQueue Mechanism: `RetryEngine.java` recalculates `next_retry_at` and schedules the task back into the priority queue, avoiding resonance wave spikes against downstream systems.

7. Dead Letter Queue (DLQ) & Poison Pill Isolation

Definition: A containment queue for malformed tasks (poison pills) that crash workers or exceed maximum retry thresholds (`maxRetries = 5`).

OptiQueue Mechanism: When retries exhaust, jobs transition to `DEAD_LETTER` . The stack trace is saved in `last_error` , and the worker thread returns safely to the pool. Operators can inspect the payload and invoke `POST /api/jobs/dead-letter/{id}/retry` to replay the task.

8. Sliding Window Rate Limiting (Redis Sorted Sets)

Definition: Rolling time window traffic throttling that eliminates the burst vulnerability of fixed-window algorithms.

OptiQueue Mechanism: `RedisRateLimiter.java` uses Redis `ZSET` : 1. Remove expired entries: `ZREMRANGEBYSCORE key 0 (now - window)`
2. Count current elements: `ZCARD key`
3. If count ≥ limit, reject with HTTP 429; else `ZADD key now uuid` and allow request.

3. Distributed Systems Interview Masterclass (Q&A)

Q1: How does OptiQueue prevent two workers from picking up the exact same job concurrently?

Answer: OptiQueue utilizes a multi-stage defense:

1. **In-Memory Thread Safety:** The `JobPriorityQueue` is guarded by a `ReentrantLock` ensuring only one consumer thread polls a job at a time.
2. **Distributed Mutual Exclusion:** Before execution, the worker acquires an exclusive Redis lock on `lock:job:{jobId}` using an atomic `SET NX PX 30000`.
3. **Optimistic Versioning in PostgreSQL:** State transition uses `UPDATE jobs SET status = 'IN_PROGRESS', version = version + 1 WHERE id = :id AND status = 'PENDING'`. If another worker already claimed it, the update affects 0 rows and the second worker drops the task.

Q2: What happens if a worker crashes or encounters an Out-Of-Memory (OOM) error midway through execution?

Answer:

1. The crashed worker abruptly stops sending heartbeats to Redis (`worker:heartbeat:{workerId}`).
2. Within 15 seconds, the Redis heartbeat key expires via TTL.
3. The `FailoverDetector` background thread detects an orphaned job in state `IN_PROGRESS` with no matching heartbeat.
4. The supervisor increments the retry counter, marks the job `PENDING`, logs the diagnostic error, and pushes the job back to the Min-Heap for an active worker to execute.

Q3: Why is Jitter strictly required in Exponential Backoff?

Answer: If an external service experiences an outage, hundreds of failed tasks without jitter would calculate identical retry delays (e.g., exactly 2s, 4s, 8s). All workers would retry simultaneously in synchronized bursts (**Thundering Herd Problem**), repeatedly crashing the recovering dependency. Adding random jitter disperses retries evenly across time, smoothing the load.

Q4: How does OptiQueue achieve Exactly-Once semantics?

Answer: In distributed systems over real networks, physical exactly-once transmission is impossible. OptiQueue guarantees **effectively exactly-once execution** through *At-Least-Once Delivery* paired with *Idempotent Consumer Processing*. Every job has an `idempotencyKey` backed by a database unique constraint, and handlers verify idempotency states before dispatching side-effects.

Q5: Explain the Fencing Token problem in distributed locking and how OptiQueue mitigates it.

Answer: If Worker A holds a 30s Redis lock and experiences a 35s JVM Stop-the-World garbage collection pause, its lock expires. Worker B claims the lock. Worker A awakens and writes to the DB unaware its lease lapsed. OptiQueue prevents this using PostgreSQL optimistic version numbers as **fencing tokens**. When Worker B claims the task, it increments the version from V1 to V2. When Worker A tries to commit using V1, the update fails.

Q6: How does OptiQueue isolate Poison Pills and prevent cascading worker pool death?

Answer: A defensive `try-catch(Throwable)` barrier inside the worker consumer loop intercepts both checked exceptions and fatal JVM errors. Unrecoverable failures increment the retry count; once `maxRetries = 5` is exceeded, the job is evicted to `DEAD_LETTER` with full stack trace capture. The worker thread is returned unharmed to the pool.

Q7: Walk through what happens under the hood when "Launch Demo Workload" is clicked.

Answer:

1. The dashboard calls `POST /api/jobs/demo-batch` with the authorized `X-API-Key`.
2. The controller generates 10 heterogeneous tasks (4 Email, 3 Webhook, 2 Report Gen, 1 intentional failure DLQ test) with distinct priorities (P1-P5).
3. Jobs are durably saved to Neon PostgreSQL in state `PENDING` and queued in the Min-Heap.
4. Worker threads lease tasks, acquire Redis locks, execute business handlers, and record execution results.
5. 9 tasks complete; 1 task retries with jitter and quarantines into the Dead Letter Queue. The dashboard animates the metrics live in real time.

4. Mathematical Formulas & Whiteboard Reference

Formula Name	Mathematical Expression	Application in OptiQueue
Little's Law	`L = λ × W`	Determines worker pool sizing: Concurrency (L) = Ingestion Rate (λ) × Average Service Time (W).
Amdahl's Law	`S(N) = 1 / ((1 - P) + (P / N))`	Calculates theoretical speedup with N parallel worker threads given parallelizable fraction P.
Decorrelated Jitter	`Delay = rand(0, min(MaxDelay, Base × 2^attempt))`	Used in `RetryEngine.java` to disperse retries across time and prevent Thundering Herd spikes.
Min-Heap Priority Score	`Score = (Priority × 10^12) + DeadlineMillis`	Orders tasks in `JobPriorityQueue.java` by priority and earliest SLA deadline.

5. Production Deployment Telemetry & Architecture Limits

- **Live Web App:** https://optiqueue.onrender.com
- **Compute:** Render Web Service (Spring Boot 3 / OpenJDK 17 Docker container).
- **Relational DB:** Neon Serverless PostgreSQL 16 (HikariCP connection pool, max-connections: 10).
- **Distributed Cache:** Upstash Serverless Redis 8.4 (Sub-millisecond latency, distributed locks, rate limiter).
- **Total Infrastructure Cost: $0.00 / month (100% Free Tier Architecture)**.

— OptiQueue Distributed Task Scheduling & Execution Engine —
Prepared for Technical & System Design Interviews • 2026